# FIN-RSI: An Agent-Native Library of Financial Skills, Checks, and Self-Improvement

**Anonymous submission**


## Abstract

Financial research code fails silently: a program that applies stock splits twice, or trades on the closing price it reads, still runs and still returns a backtest. We present FIN-RSI, an agent-native Python library that puts three things behind interfaces a language-model agent can call: *skills* (131 dated skill documents, 48 algorithms and 6 rule cards), *checks* (39 guards, including property checks that need no expected output) and *self-improvement* (7 methods behind one `fit`/`solve`/`state` interface with metered, equal budgets). With it comes a suite of 34 financial programming tasks on synthetic and real prices, each with an independent reference and a hidden grader that no method sees. [One sentence of results.] The library, tasks and protocol will be released.


## 1 Introduction

Financial research code fails silently. A program that applies stock splits twice, decides a position with the close it trades over, or leaves warm-up rows missing still runs and still returns a backtest. Language-model agents now write such code (Zhang et al. 2024; Bigeard et al. 2025), and they make these mistakes: in our earlier audit, all 24 programs a 14B coding model wrote re-applied split factors that the prompt defined as already cumulative.

An agent would need three things to avoid them: knowledge of the failure modes, checks that detect each failure without an expected output, and a way to turn check results into better programs. All three exist, but separately. Skill documents and tool collections give knowledge; testing utilities give checks; self-improvement methods each come with their own tasks, feedback and budgets (Shinn et al. 2023; Chen et al. 2024; Zhao et al. 2024; Agrawal et al. 2026), so their results cannot be compared. Intrinsic self-correction without an external signal rarely helps (Huang et al. 2024), and financial benchmarks score answers or returns (Xie et al. 2024; Chen et al. 2025); a return can reward a wrong program on a favourable price path, and repeated trials make such false discoveries likely (Bailey et al. 2017).

FIN-RSI puts all three behind interfaces an agent can call (Figure 1). PyOD put outlier detectors behind one interface (Zhao et al. 2019), which let ADBench compare them on equal terms (Han et al. 2022). FIN-RSI does the same for self-improvement: every method gets the same judge and the same metered budget. A user connects a model once, as one `generate(messages)` callable. Our contributions:

- an agent-native library of skills, checks and tools (§2);
- 7 self-improvement methods behind one interface with equal budgets (§3);
- 34 financial programming tasks with independent references, a hidden grader, an algorithm-level held-out split, real-price panels and an ablation that separates skills, checks and self-improvement (§4).

## 2 The FIN-RSI library

FIN-RSI is a Python package whose base install needs only numpy, pandas and scipy.

**Skills.** 131 skill documents in 17 plugins follow the Agent Skills specification (Agent Skills 2026), with 75 reference files and 125 runnable scripts. A skill records the defaults, licence changes and silent failure modes that a model's training prior gets wrong; each claim is dated and marked by how it was verified. A further 92 federated packs (2,390 skills, off by default, not verified here) include 100 investor templates built on named investors' approaches. A catalog of 48 algorithms covers 11 task types (25 native, 23 adapters to 13 optional libraries, reported as missing rather than replaced when absent). `positions()` takes raw quotes and split factors, applies the factors once and returns positions decided on the previous close; `rolling()` does the same for volatility, risk and forecasts. Between documents and code sit 6 rule cards in 3 families (timing, corporate actions, data hygiene; Appendix Table 2). A card holds a short rule, the skill passage it comes from, the checks that verify it and, where one exists, the library call that follows it. A task declares traits such as *split factors* and receives only the cards they select.

**Checks.** The library has 39 guards behind one interface, which returns whether the guard passed, a message and warnings; 32 take JSON inputs and are exported as tools. For generated programs, three property checks need no expected output:

- `finite_output`: no missing or infinite values;

This is an anonymized submission for review purposes only. Distribution, citation, or public sharing of this manuscript is strictly prohibited. Copyright and publication details will appear in the final version if accepted.

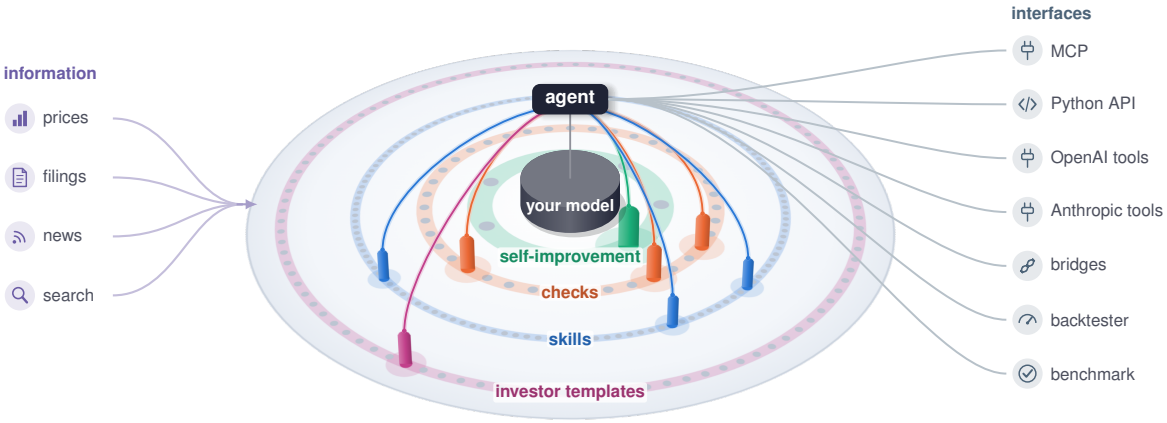



Figure 1: FIN-RSI at a glance. The user's model sits at the centre of four rings, one pin per item: the 7 self-improvement methods, the 39 checks, the 131 skills and the 100 investor templates. An agent is the model plus the pins that stand up: one method and any checks, skills and templates. Information comes in on the left; the agent is used through the interfaces on the right.

- `split_representation`: re-run on the same economic prices, given as adjusted closes with unit factors, the output must not change ($10^{-8}$);
- `execution_lag`: perturb prices from row $k$ on; no position up to row $k$ may move. A rule that trades on the close it reads fails, although it passes a plain causality test.

These are metamorphic relations (Segura et al. 2016) for financial code. Each check carries a *cause*, one sentence on what it observed, and `explain()` pairs it with the cards that address it. We count the cause as part of the check and the card as part of the skills.

**Agent interfaces.** One registry yields 67 tools under the same names as a Model Context Protocol server (Model Context Protocol 2026) and as OpenAI and Anthropic tool schemas: read and search skills, run algorithms, run guards. `agents.Team` turns the user's `generate` callable into a writer and one reviewer per rule family; only the checks decide acceptance.

## 3 Self-improvement behind one interface

Every method is a class with one constructor and three calls:

```
m = rsi.Reflexion(library=True)
m.fit(generate, tasks.dev)
ep = m.solve(generate, task, seed=11)
ep.accepted, ep.code, ep.usage
m.state()
```

`state()` returns what the method learned and is hashed into every episode record; `fit()` does nothing for a method that does not learn. `rsi.benchmark()` fits each method once on the development tasks and runs it on every held-out task and seed.

**What a method sees.** `solve()` gets the task text and a judge on which it can only call `check(code)`. That returns whether the program ran, its error, each check's result and the causes of the failed ones, never the grade (Figure 2). Learning methods may use development grades to keep or reject a candidate, but no text a model reads contains a grade.

**Equal budgets.** A meter refuses any call beyond the budget and records which limit stopped the episode. Each episode gets 3 program executions, 6 model calls (reflections and reviews included) and 12,000 generated tokens. Learning methods also get 48 development episodes, 6 proposals, 600 calls and 1,200,000 tokens.

**Methods.** The 7 methods (Appendix Table 1) keep the mechanism each is known for: `NoImprove` (one program); `SelfDebug`, which returns only errors (Chen et al. 2024); `CheckFeedback`, which returns the failed checks with causes and cards; `Reflexion`, which adds a written reflection before each retry (Shinn et al. 2023); `Team`, with one reviewer per rule family; `Evolve`, which evolves a prompt addendum on development tasks (Agrawal et al. 2026); and `PlaybookLearning` (Zhao et al. 2024; Suzgun et al. 2026). The playbook proposer sees development traces without grades, and one proposal edits one entry. A candidate is accepted only if its score does not drop on either of two disjoint halves of the development tasks, rises on one, and no task that passed every check before now fails one (the *no-regression* rule; Appendix Figure 4).

**Skill arms.** Two switches split the skills: `cards` adds the task's rule cards to the prompt and the feedback; `library` adds the interface text and allows `import fin_skills`. Their 4 combinations (`none`, `cards`, `library`, `cards+library`) all keep the task definition and see the failed checks with their causes. Within an arm, `NoImprove` versus `CheckFeedback` measures the checks and `CheckFeedback` versus `PlaybookLearning` measures self-improvement; for one method, comparing arms measures the skills (Appendix Figure 5).

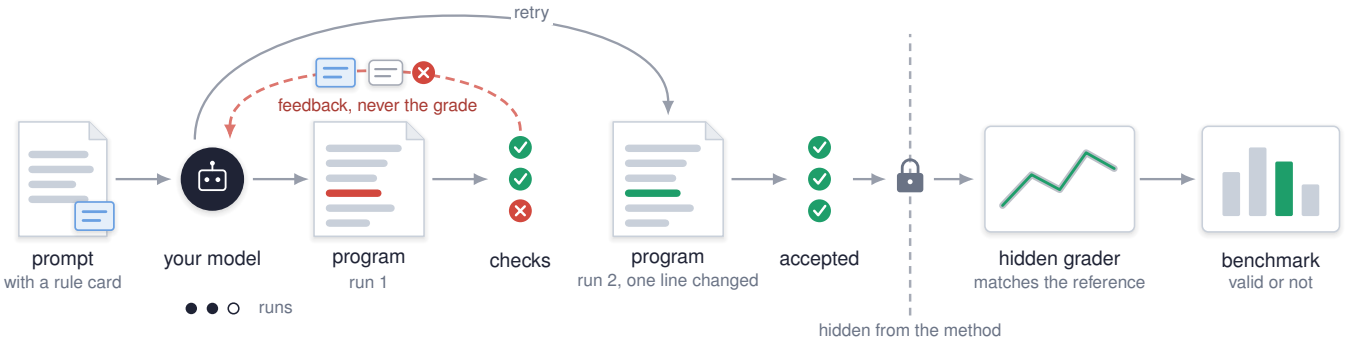



Figure 2: One episode of `momentum_a`. The first program reads raw quotes on one line (red) and fails `split_representation`; the method gets the failed check, its cause and the `split_once` card, never the grade, and retries within the budget of 3 runs. The second program changes that line (green) and passes all three checks. Only then, behind the wall, does the hidden grader compare it with the reference, and its verdict goes to the benchmark alone. These are the judge's real verdicts for the two programs, on every seed and on the real panel.

## 4 Task suite and protocol

34 tasks cover 17 algorithms in 5 families, two variants each (Appendix Table 3). A task states a rule in prose; the model implements `solve(prices, split_factors)` returning positions, or for volatility, risk and forecast tasks the value known at the previous close. Each task can be solved by hand or by one library call.

**Data.** Synthetic panels have four assets and 128 business days of seeded Gaussian returns, with four splits at fixed rows (2-for-1, 3-for-1, 3-for-2, 1-for-2), on 3 seeds. Variant a gives cumulative factors in alphabetical order; variant b gives event ratios, shuffled columns and other parameters. Held-out tasks also run on a real panel: as-traded closes of Amazon, Shopify, Alphabet and Tesla for 128 sessions from 31 March 2022 to 3 October 2022, in which all four split (20-, 10-, 20- and 3-for-1; Appendix Figure 6). Yahoo Finance serves the closes split-adjusted; we rebuild as-traded quotes from a split table checked against the companies' announcements and published pre-split closes, and release the fetch code and the file's SHA-256, since the prices may not be redistributed.

**Judge and hidden grader.** The judge runs each program in a separate process with a memory limit and a 120-second timeout (in our runs also confined with Landlock and seccomp), refuses `import fin_skills` without library access, and applies the three checks of §2 (Figure 3). A program is *valid* only if it ran, passed every check and matches an independent numpy reference within $10^{-8}$ ($10^{-5}$ for `min_variance`). A program that passes the checks but is wrong is *accepted but wrong*; we report that rate because the checks cannot see it.

**Qualification.** Before any model ran: on every task and data panel, the reference agrees with the library to within $2.8 \times 10^{-14}$ ($1.4 \times 10^{-6}$ for `min_variance`); a correct program passes every check and is graded valid; and on a position task and a table task, planted versions of each failure (same close, no adjustment, factors cumulated twice, missing warm-up) fail their check and the grade. Building the references exposed a library defect: the long-only minimum-variance optimiser stopped early on daily-scale covariances, off by up to $1.2 \times 10^{-2}$, until we rescaled the covariance before solving.

**Split and protocol.** The split is by algorithm. The 5 algorithms a model has already seen go to development, and a seeded draw sends 4 of the other 12 there too: 18 development and 16 held-out tasks, the latter covering all 5 families. The primary outcome is the held-out valid rate, with task-clustered bootstrap intervals and Holm-corrected paired comparisons. The main comparison runs 7 methods $\times$ 2 arms $\times$ 16 tasks $\times$ 3 seeds $\times$ 5 open-weight models (3,360 episodes); the real panel adds 1,120, reported separately, and the skill-arm ablation 1,440.

## 5 Preliminary results

[Model qualification on two development tasks (which candidate models load, return a program that runs, and at what cost per episode), then held-out valid rates per method and arm, with task-clustered 95% intervals and the accepted-but-wrong rate, for the models whose runs finish before the deadline. The full matrix goes to the extended version.]

**Limitations.** Most episodes use synthetic panels with a fixed set of pitfalls; the real panel is one window of four stocks with forward splits only and no dividends. The checks reject only the failures they were written for, hence the accepted-but-wrong rate. All models are open-weight, under one attempt budget.

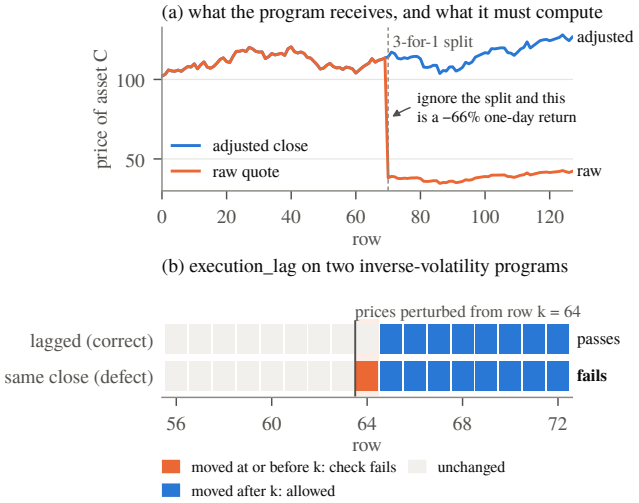



Figure 3: (a) Raw and adjusted closes of one asset on a synthetic panel. (b) `execution_lag` perturbs prices from row $k = 64$: lagged positions first move at row 65 and pass; positions read on the close they trade over move at row 64 and fail.

## References


Agent Skills. 2026. Agent skills specification. https://agentskills.io/specification. Accessed 2026-11-04.

Lakshya A Agrawal, Shangyin Tan, Dilara Soylu, Noah Ziems, Rishi Khare, Krista Opsahl-Ong, Arnav Singhvi, Herumb Shandilya, Michael J Ryan, Meng Jiang, Christopher Potts, Koushik Sen, et al. 2026. GEPA: Reflective prompt evolution can outperform reinforcement learning. In *International Conference on Learning Representations (ICLR)*.

David H Bailey, Jonathan Borwein, Marcos López de Prado, and Qiji Jim Zhu. 2017. The probability of backtest overfitting. *Journal of Computational Finance*, 20(4):39–69.

Antoine Bigeard, Langston Nashold, Rayan Krishnan, and Shirley Wu. 2025. Finance agent benchmark: Benchmarking llms on real-world financial research tasks. *arXiv preprint arXiv:2508.00828*.

Xinyun Chen, Maxwell Lin, Nathanael Schärli, and Denny Zhou. 2024. Teaching large language models to self-debug. In *International Conference on Learning Representations (ICLR)*.

Yanxu Chen, Zijun Yao, Yantao Liu, Amy Xin, Jin Ye, Jianing Yu, Lei Hou, and Juanzi Li. 2025. StockBench: Can LLM agents trade stocks profitably in real-world markets? *arXiv preprint arXiv:2510.02209*.

Zhibin Gou, Zhihong Shao, Yeyun Gong, Yelong Shen, Yujiu Yang, Nan Duan, and Weizhu Chen. 2024. CRITIC: Large language models can self-correct with tool-interactive critiquing. In *International Conference on Learning Representations (ICLR)*.

Songqiao Han, Xiyang Hu, Hailiang Huang, Minqi Jiang, and Yue Zhao. 2022. ADBench: Anomaly detection benchmark. In *Advances in Neural Information Processing Systems 35 (NeurIPS 2022), Datasets and Benchmarks Track*.

Jie Huang, Xinyun Chen, Swaroop Mishra, Huaixiu Steven Zheng, Adams Wei Yu, Xinying Song, and Denny Zhou. 2024. Large language models cannot self-correct reasoning yet. In *International Conference on Learning Representations (ICLR)*.

Marcos López de Prado. 2018. *Advances in Financial Machine Learning*. John Wiley & Sons.

Aman Madaan, Niket Tandon, Prakhar Gupta, Skyler Hallinan, Luyu Gao, Sarah Wiegreffe, Uri Alon, Nouha Dziri, Shrimai Prabhumoye, Yiming Yang, Shashank Gupta, Bodhisattwa Prasad Majumder, Katherine Hermann, Sean Welleck, Amir Yazdanbakhsh, and Peter Clark. 2023. Self-refine: Iterative refinement with self-feedback. In *Advances in Neural Information Processing Systems 36 (NeurIPS 2023)*.

Model Context Protocol. 2026. Model context protocol specification, revision 2026-07-28. https://modelcontextprotocol.io/specification.

Marco Tulio Ribeiro, Tongshuang Wu, Carlos Guestrin, and Sameer Singh. 2020. Beyond accuracy: Behavioral testing of NLP models with CheckList. In *Proceedings of the 58th Annual Meeting of the Association for Computational Linguistics (ACL)*, pages 4902–4912.

Sergio Segura, Gordon Fraser, Ana B. Sánchez, and Antonio Ruiz-Cortés. 2016. A survey on metamorphic testing. *IEEE Transactions on Software Engineering*, 42:805–824.

Noah Shinn, Federico Cassano, Ashwin Gopinath, Karthik Narasimhan, and Shunyu Yao. 2023. Reflexion: Language agents with verbal reinforcement learning. In *Advances in Neural Information Processing Systems 36 (NeurIPS 2023)*.

Mirac Suzgun, Mert Yuksekgonul, Federico Bianchi, Dan Jurafsky, and James Zou. 2026. Dynamic cheatsheet: Test-time learning with adaptive memory. In *Proceedings of the 19th Conference of the European Chapter of the Association for Computational Linguistics (Volume 1: Long Papers)*.

Guanzhi Wang, Yuqi Xie, Yunfan Jiang, Ajay Mandlekar, Chaowei Xiao, Yuke Zhu, Linxi Fan, and Anima Anandkumar. 2024. Voyager: An open-ended embodied agent with large language models. *Transactions on Machine Learning Research*.

Qianqian Xie, Weiguang Han, Zhengyu Chen, Ruoyu Xiang, Xiao Zhang, Yueru He, Mengxi Xiao, Dong Li, Yongfu Dai, Duanyu Feng, Yijing Xu, et al. 2024. FinBen: A holistic financial benchmark for large language models. In *Advances in Neural Information Processing Systems 37 (NeurIPS 2024), Datasets and Benchmarks Track*.

Wentao Zhang, Lingxuan Zhao, Haochong Xia, Shuo Sun, Jiaze Sun, Molei Qin, Xinyi Li, Yuqing Zhao, Yilei Zhao, Xinyu Cai, Longtao Zheng, Xinrun Wang, and Bo An. 2024. A multimodal foundation agent for financial trading: Tool-augmented, diversified, and generalist. *arXiv preprint arXiv:2402.18485*.

Andrew Zhao, Daniel Huang, Quentin Xu, Matthieu Lin, Yong-Jin Liu, and Gao Huang. 2024. ExpeL: LLM agents are

experiential learners. In *Proceedings of the AAAI Conference on Artificial Intelligence*, volume 38, pages 19632–19642.

Yue Zhao, Zain Nasrullah, and Zheng Li. 2019. PyOD: A Python toolbox for scalable outlier detection. *Journal of Machine Learning Research (JMLR MLOSS)*, 20(96):1–7.

## A Self-improvement methods

`Reflexion` asks for a reflection of at most five sentences, without code, and keeps the reflections for the rest of the episode. In `Team`, a reviewer is asked only when a check of its own family failed; it sees the program, the failed checks, their causes and its family's cards, and answers with a diagnosis rather than code. `Evolve` rewrites a prompt addendum of at most 1,200 characters from the visible failures of the current best addendum, and keeps a candidate only if it scores strictly higher on the same development tasks. The playbook is a list of short entries on three surfaces (tool usage, retrieval, experience). Its loop rejects whitespace-only and repeated edits, and appends every proposal, evaluation and decision to a journal whose receipts carry evidence hashes; a frozen playbook cannot change. A stricter *all-pass* rule, which vetoes any candidate while a single task fails, blocks learning for a weak model, so we use no-regression.

## B Experiment design

## C Real-price panel

## D Rule cards

## E Task suite

| Method | At test time | Learned on development tasks | In the spirit of |
|---|---|---|---|
| `NoImprove` | one program, no feedback | – | baseline |
| `SelfDebug` | retry with the error only; stop once the program runs | – | Chen et al. (2024) |
| `CheckFeedback` | retry with the failed checks, their causes and cards | – | verifier feedback |
| `Reflexion` | as `CheckFeedback`, plus a written reflection before each retry | – | Shinn et al. (2023) |
| `PlaybookLearning` | as `CheckFeedback`, with a frozen playbook in the prompt | playbook entries, through the gated loop | Zhao et al. (2024); Suzgun et al. (2026) |
| `Team` | a writer and one reviewer per rule family, from the same model | – | multi-role review |
| `Evolve` | as `CheckFeedback`, with an evolved prompt addendum | the addendum, over 3 generations on 4 tasks | Agrawal et al. (2026) |

Table 1: The 7 self-improvement methods. All share the prompt, the judge and the budget; each keeps the mechanism it is known for. They are not reproductions of the original systems.

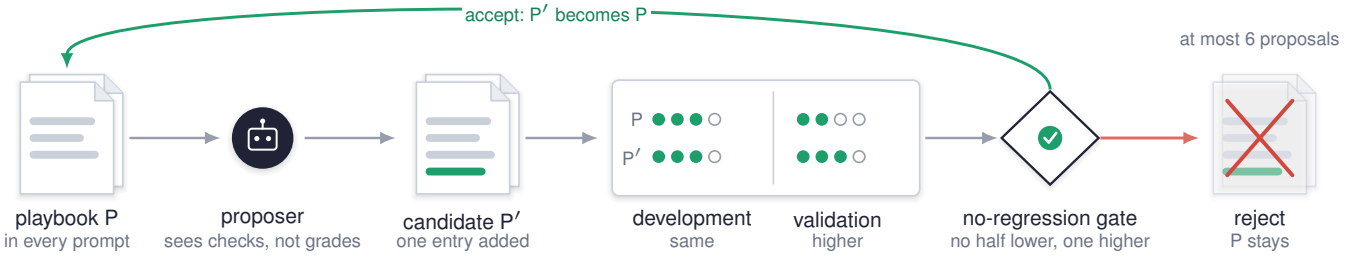



Figure 4: The gated playbook loop of `PlaybookLearning`. The proposer reads development traces without grades and adds one entry; the current and the candidate playbook are scored on two disjoint halves of the development tasks (dots: tasks that passed their checks; an example, not results). The no-regression gate accepts the candidate only if neither half is lower, one is higher and no task that passed before now fails.

| Card | Verified by | Call |
|---|---|---|
| *timing* | | |
| `execution_lag` | `execution_lag` | yes |
| `feature_causality` | `assert_causal` | – |
| *corporate actions* | | |
| `split_once` | `split_representation` `adjustment_check` | yes |
| `same_convention` | `split_representation` | – |
| *data hygiene* | | |
| `warmup` | `finite_output` | – |
| `first_return` | `finite_output` | – |

Table 2: Rule cards by family, the checks that verify each one, and whether the card names a library call that follows the rule.

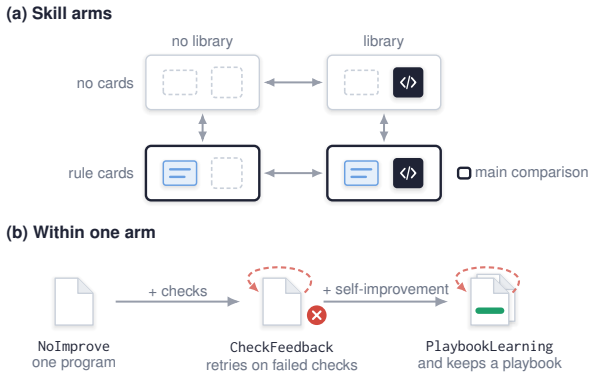



Figure 5: (a) The four skill arms: a blue card for the rule cards, a dark square for the library interface, a dashed outline for what an arm leaves out. Bold frames are the arms of the main comparison; arrows join arms that differ in one component. (b) Within one arm, `NoImprove` versus `CheckFeedback` measures the checks and `CheckFeedback` versus `PlaybookLearning` measures self-improvement.

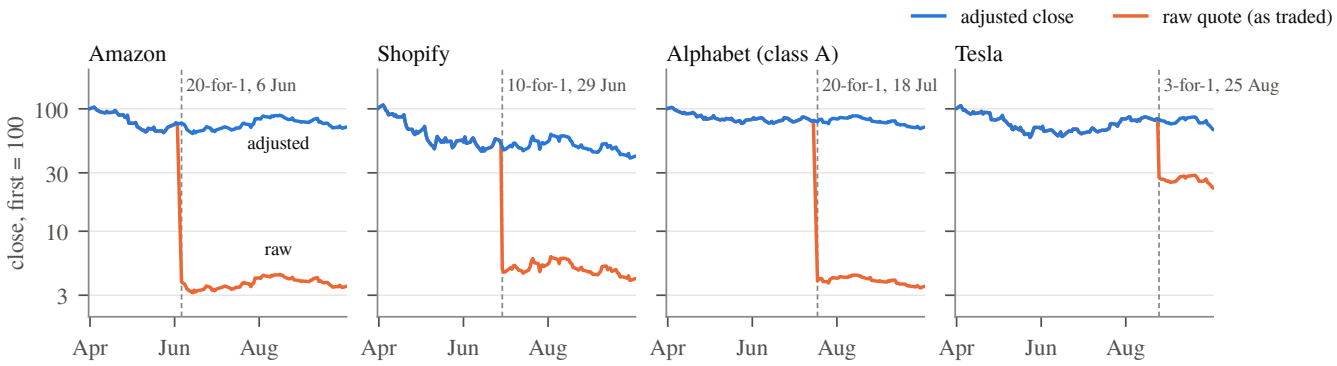



Figure 6: The real-price panel: raw quotes (as traded) and adjusted closes of the four stocks, each indexed to 100 at the first close, on a log scale. Every stock splits inside the window; a program that ignores the split factors sees a one-day loss of 67% to 95%.

| Algorithm | Split | Library call |
|---|---|---|
| *portfolio* | | |
| `cross_sectional_momentum` | dev | `positions()` |
| `hrp` | heldout | `positions()` |
| `inverse_volatility` | dev | `positions()` |
| `min_variance` | heldout | `positions()` |
| *signal* | | |
| `bollinger_reversion` | dev | `positions()` |
| `donchian_breakout` | heldout | `positions()` |
| `ma_crossover` | dev | `positions()` |
| `macd` | dev | `positions()` |
| `momentum` | dev | `positions()` |
| `rsi_reversion` | heldout | `positions()` |
| `vol_target_momentum` | dev | `positions()` |
| *volatility* | | |
| `ewma_volatility` | heldout | `rolling()` |
| `historical_volatility` | dev | `rolling()` |
| *risk* | | |
| `historical_var_es` | heldout | `rolling()` |
| `normal_var_es` | heldout | `rolling()` |
| *forecast* | | |
| `drift` | heldout | `rolling()` |
| `mean` | dev | `rolling()` |

Table 3: The 17 algorithms of the task suite, with their split and the one-step library call that solves them. Each algorithm has two tasks (variants a and b).